АВТОНОМНАЯ НЕКОММЕРЧЕСКАЯ ОРГАНИЗАЦИЯ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ УНИВЕРСИТЕТ “СИНЕРГИЯ”»

Факультет / Институт	Кафедра ИМиИКТ им. В. В. Дика
Направление подготовки	Информационные технологии и программирование
Форма обучения	Очная

Отчет по практической работе №1

На тему	Разработка структуры проекта
По дисциплине	Инструментальные средства разработки программного обеспечения

Обучающийся	Ульченков Егор Витальевич	
Группа	VДКИП-111прог	

Преподаватель	Сибирев И. В.	

Москва 2026 г.

Содержание

Цель работы....2
Контрольные вопросы....2
Практическая часть....3
Техническое задание....3
Введение....3
Основание для разработки....3
Назначение....3
Требования к программе....4
Функциональные....4
Технические....4
Входные и выходные данные....4
Алгоритм вычислений....4
Требования к интерфейсу (WPF)....4
Тестирование....5
Реализация....5
Листинг программы....5
Тестовый запуск....8
Выводы....8

Цель работы

Изучение этапов проектирования программного обеспечения и приобретение практических навыков разработки структуры проекта и технического задания в среде Visual Studio 2026.

Контрольные вопросы

1. **Перечислите этапы разработки программных продуктов.**

   Постановка задачи, ТЗ, проектирование, кодирование, тестирование, внедрение.

2. **Для чего необходимо техническое задание?**

   Чтобы зафиксировать требования к функционалу и избежать разногласий между заказчиком и разработчиком.

3. **Кто занимается разработкой технического задания?**

   Обычно системный аналитик или разработчик на основе требований заказчика.

4. **Какие пункты включает техническое задание?**

   Введение, основание, назначение, требования к ПО и документации.

Практическая часть

Вариант 8

8. Дана вещественная матрица размерности n * m. Сформировать вектор b, в котором элементы вычисляются как:

1. произведение элементов соответствующих строк;
2. среднее арифметическое соответствующих столбцов;
3. разность наибольших и наименьших элементов соответствующих строк;
4. значения первых отрицательных элементов в столбце.

Техническое задание

Введение

Целью разработки является создание WPF программы для вычисления вектора B

Основание для разработки

Задание к лабораторной работе №1.

Назначение

Программа предназначена для ввода вещественной матрицы произвольного размера, автоматического расчёта четырёх вариантов вектора по заданным формулам и визуализации результатов в графическом интерфейсе.

Требования к программе

Функциональные

Ввод размерности (n, m) и элементов матрицы; валидация данных (тип, диапазон, знаки); расчёт 4-х векторов; обработка граничных случаев (отсутствие отрицательных элементов, нули, переполнение произведения).

Технические

C#, .NET 10, WPF.

Входные и выходные данные

- Вход: целые n, m >= 1; матрица A[n×m] (double).
- Выход: векторы b1 (длина n), b2 (длина m), b3 (длина n), b4 (длина m). При отсутствии отрицательного элемента в столбце — значение null или текст «нет».

Алгоритм вычислений

1. Проверить корректность n, m и элементов матрицы.
2. $b_{1i} = \prod_j A_{ij}$ (i = 0..n-1).
3. $b_{2j} = (\sum_i A_{ij}) / n$ (j = 0..m-1).
4. $b_{3i} = \max_j(A_{ij}) - \min_j(A_{ij})$ (i = 0..n-1).
5. $b_{4j} = A_{kj}$, где k — минимальный индекс с $A_{kj} < 0$. Если таких нет, $b_{4j} = \text{null}$.
6. Отформатировать и вывести результаты.

Требования к интерфейсу (WPF)

1. Поля ввода n, m + кнопка «Создать матрицу».
2. DataGrid для ввода/отображения A.
3. Кнопки: «Рассчитать», «Очистить».

4. 4 блока вывода результатов (таблицы или списки) с подписями, соответствующими вариантам 1–4.

Тестирование

1. Корректный ввод произвольной матрицы.
2. Вырожденные размеры (1×1, 1×m, n×1).
3. Столбцы без отрицательных элементов.
4. Некорректный ввод (текст, пустые поля, отрицательные размеры).
5. Проверка точности и переполнения при больших/малых значениях.

Реализация

Листинг программы

MainWindow.xaml

```
<Window x:Class="MatrixCalc.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Вычисление вектора B" Height="450" Width="800">
    <DockPanel Margin="10">
        <StackPanel Orientation="Horizontal" DockPanel.Dock="Top"
Margin="0,0,0,10">
            <TextBlock Text="N:" VerticalAlignment="Center"/>
            <TextBox x:Name="tbN" Width="40" Text="3" Margin="5,0"/>
            <TextBlock Text="M:" VerticalAlignment="Center"
Margin="10,0,0,0"/>
            <TextBox x:Name="tbM" Width="40" Text="3" Margin="5,0"/>
            <Button Content="Создать" Click="CreateMatrix_Click"
Margin="10,0,5,0"/>
            <Button Content="Заполнить случайно" Click="FillRandom_Click"
Margin="5,0"/>
            <Button Content="Рассчитать" Click="Calculate_Click"
Margin="5,0"/>
            <Button Content="Очистить" Click="Clear_Click" Margin="5,0"/>
        </StackPanel>
        <Grid>
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*"/>
                <ColumnDefinition Width="*"/>
            </Grid.ColumnDefinitions>
            <DataGrid x:Name="dgMatrix" AutoGenerateColumns="False"
Margin="0,0,10,0"
                      CanUserAddRows="False" CanUserDeleteRows="False"
HeadersVisibility="Column"/>
            <StackPanel Grid.Column="1">
                <TextBlock Text="Результаты:" FontWeight="Bold"
Margin="0,0,0,5"/>
```

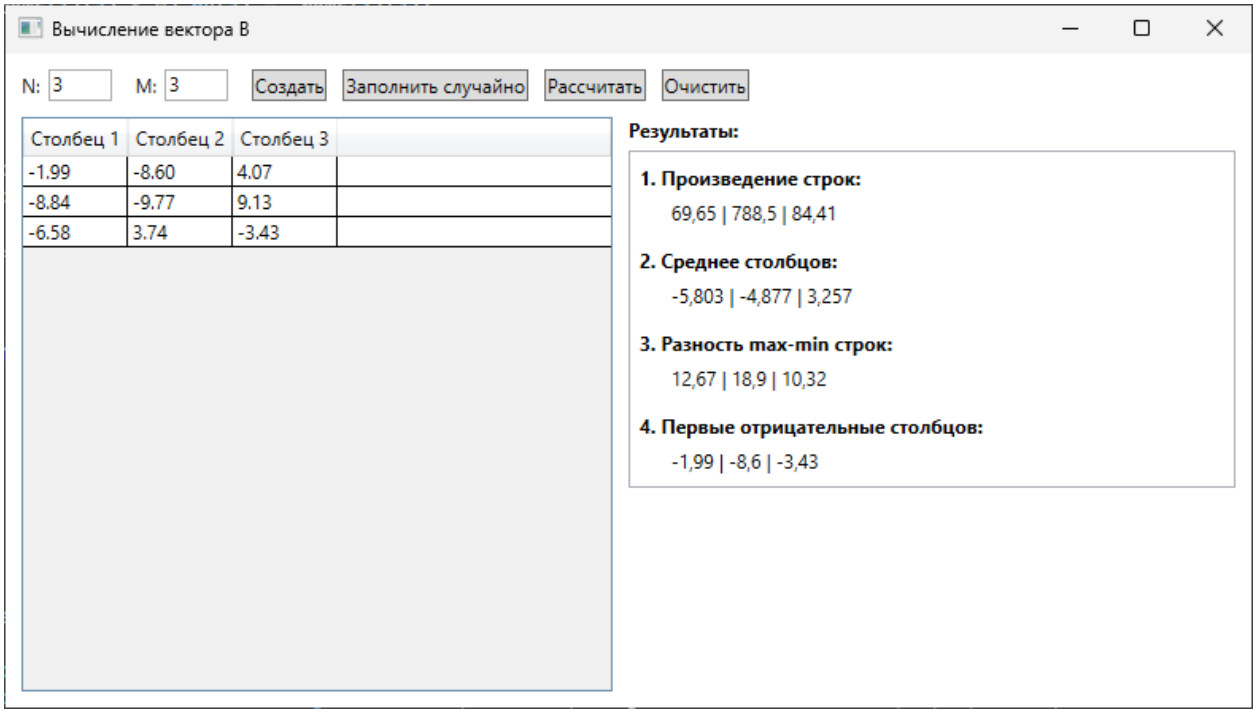
```
                <ListBox x:Name="lbResults"
ScrollViewer.HorizontalScrollBarVisibility="Auto"/>
            </StackPanel>
        </Grid>
    </DockPanel>
</Window>
```

MainWindow.xaml.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Windows;
using System.Windows.Controls;
using System.Windows.Data;
namespace MatrixCalc { public partial class MainWindow : Window { private int
_n, _m; private List<double[]> _rows;
    public MainWindow() => InitializeComponent();

    private void CreateMatrix_Click(object sender, RoutedEventArgs e)
    {
        if (!int.TryParse(tbN.Text, out _n) || !int.TryParse(tbM.Text, out
_m) || _n < 1 || _m < 1)
        {
            MessageBox.Show("Введите корректные размеры N, M >= 1.");
            return;
        }

        _rows = new List<double[]>();
        for (int i = 0; i < _n; i++) _rows.Add(new double[_m]);

        dgMatrix.Columns.Clear();
        for (int j = 0; j < _m; j++)
            dgMatrix.Columns.Add(new DataGridTextColumn
            {
                Header = $"Столбец {j + 1}",
                Binding = new Binding($"[{j}]") { Mode = BindingMode.TwoWay,
StringFormat = "F2" }
            });

        dgMatrix.ItemsSource = _rows;
        lbResults.Items.Clear();
    }

    private void FillRandom_Click(object sender, RoutedEventArgs e)
    {
        if (_rows == null) { MessageBox.Show("Сначала создайте матрицу.");
return; }
        var rnd = new Random();
        foreach (var row in _rows)
            for (int j = 0; j < _m; j++)
                row[j] = Math.Round(rnd.NextDouble() * 20 - 10, 2);
        dgMatrix.Items.Refresh();
    }

    private void Calculate_Click(object sender, RoutedEventArgs e)
    {
        if (_rows == null || _rows.Count != _n) { MessageBox.Show("Сначала
создайте матрицу."); return; }
        dgMatrix.CommitEdit();
```

```
        var b1 = new double[_n];
        var b2 = new double[_m];
        var b3 = new double[_n];
        var b4 = new double?[_m];

        for (int i = 0; i < _n; i++)
        {
            double prod = 1, min = _rows[i][0], max = _rows[i][0];
            foreach (double v in _rows[i])
            {
                prod *= v;
                if (v < min) min = v;
                if (v > max) max = v;
            }
            b1[i] = prod;
            b3[i] = max - min;
        }

        for (int j = 0; j < _m; j++)
        {
            double sum = 0;
            b4[j] = null;
            for (int i = 0; i < _n; i++)
            {
                sum += _rows[i][j];
                if (b4[j] == null && _rows[i][j] < 0) b4[j] = _rows[i][j];
            }
            b2[j] = sum / _n;
        }

        lbResults.Items.Clear();
        AddResult("1. Произведение строк:", b1.Select(v =>
v.ToString("G4")));
        AddResult("2. Среднее столбцов:", b2.Select(v => v.ToString("G4")));
        AddResult("3. Разность max-min строк:", b3.Select(v =>
v.ToString("G4")));
        AddResult("4. Первые отрицательные столбцов:", b4.Select(v =>
v.HasValue ? v.Value.ToString("G4") : "нет"));
    }

    private void Clear_Click(object sender, RoutedEventArgs e)
    {
        tbN.Text = tbM.Text = "3";
        dgMatrix.ItemsSource = null;
        dgMatrix.Columns.Clear();
        _rows = null;
        lbResults.Items.Clear();
    }

    private void AddResult(string title, IEnumerable<string> values)
    {
        lbResults.Items.Add(new TextBlock { Text = title, FontWeight =
FontWeights.Bold, Margin = new Thickness(0, 5, 0, 2) });
        lbResults.Items.Add(new TextBlock { Text = "   " + string.Join(" | ",
values), Margin = new Thickness(10, 0, 0, 5) });
    }
}
}
```

Тестовый запуск



Выводы

- Разработано WPF-приложение для вычисления вектора B на основе вещественной матрицы размерности n×m согласно техническому заданию.
- Реализованы все четыре алгоритма вычисления элементов вектора:
  - Произведение элементов строк матрицы
  - Среднее арифметическое столбцов
  - Разность между максимальным и минимальным элементами строк
  - Поиск первых отрицательных элементов в столбцах
- Добавлен функционал автоматического заполнения матрицы случайными значениями в диапазоне [-10; 10], что упрощает тестирование программы.

- Реализован интуитивно понятный интерфейс с использованием DataGrid для ввода/редактирования матрицы и ListBox для отображения результатов.
- Проведено тестирование программы на матрице 3×3, все вычисления выполнены корректно:
  - Произведения строк: 69,65 | 788,5 | 84,41
  - Средние значения столбцов: -5,803 | -4,877 | 3,257
  - Разности max-min: 12,67 | 18,9 | 10,32
  - Первые отрицательные элементы: -1,99 | -8,6 | -3,43
- Обеспечена обработка граничных случаев (отсутствие отрицательных элементов, проверка корректности ввода размеров матрицы).
- Приложение соответствует требованиям ТЗ и готово к использованию для решения задач матричных вычислений.